TOPSTECHNOLOGIES
Training | Outsourcing | Placement | Study Abroad

Comptia N+ Assignment

[N+ - Network security, Maintenance and Troubleshooting procedures]

Authentication and Access Control

Task:

❖ **1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.**

➢ **Aim**

- To create a simple login form for a mock Zomato Admin Panel that accepts a username and password and displays **"Access Granted"** only when both entered values match the predefined administrator credentials.

➢ **Introduction**

- Authentication is a security process used to verify the identity of a user before allowing access to a system. It ensures that only authorized users can log in.
- Access Control determines which users are permitted to access a particular system or resource.

- In this assignment, a simple login form is created for a mock Zomato Admin Panel. The form asks the administrator to enter a username and password. JavaScript checks the entered credentials against predefined sample credentials. If both match, the message "Access Granted" is displayed; otherwise, an error message is shown.

➢ **Requirements**

- Computer or laptop
- Text editor (Notepad or Visual Studio Code)
- Web browser (Google Chrome or Microsoft Edge)
- Basic knowledge of HTML, CSS, and JavaScript

➢ **Admin Login Credentials**

➢ The following sample credentials are used for this practical:

Field	Sample Value
Username	admin
Password	Zomato@123

➢ **HTML, CSS and JavaScript Code**

➢ Create a file named zomato_admin_login.html and write the following code:

```
<!DOCTYPE html>
<html>
<head>
   <title>Zomato Admin Login</title>
```

```
<style>
    body {
        font-family: Arial, sans-serif;
        background-color: #f5f5f5;
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
        margin: 0;
    }

    .login-box {
        background-color: white;
        padding: 30px;
        width: 320px;
        border-radius: 10px;
        box-shadow: 0 0 10px #cccccc;
        text-align: center;
    }

    h2 {
        color: #e23744;
    }

    input {
        width: 90%;
        padding: 10px;
        margin: 10px 0;
        border: 1px solid #cccccc;
        border-radius: 5px;
```

```
        }

        button {
            width: 97%;
            padding: 10px;
            background-color: #e23744;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        #message {
            margin-top: 15px;
            font-weight: bold;
        }
    </style>
</head>

<body>

    <div class="login-box">
        <h2>Zomato Admin Panel</h2>

        <form onsubmit="checkLogin(event)">
            <input type="text" id="username"

         placeholder="Enter Username" required>


  <input type="password" id="password"
```

```
            placeholder="Enter Password" required>

        <button type="submit">Login</button>
    </form>

    <p id="message"></p>
</div>

<script>
    function checkLogin(event) {
        event.preventDefault();

        const correctUsername = "admin";
        const correctPassword = "Zomato@123";


        const username =
            document.getElementById("username").value;

        const password =
            document.getElementById("password").value;

        const message =
            document.getElementById("message");

        if (username === correctUsername &&
            password === correctPassword) {

            message.style.color = "green";
            message.innerHTML = "Access Granted";
```

```
        } else {

            message.style.color = "red";
            message.innerHTML =
                "Invalid Username or Password";
        }

}

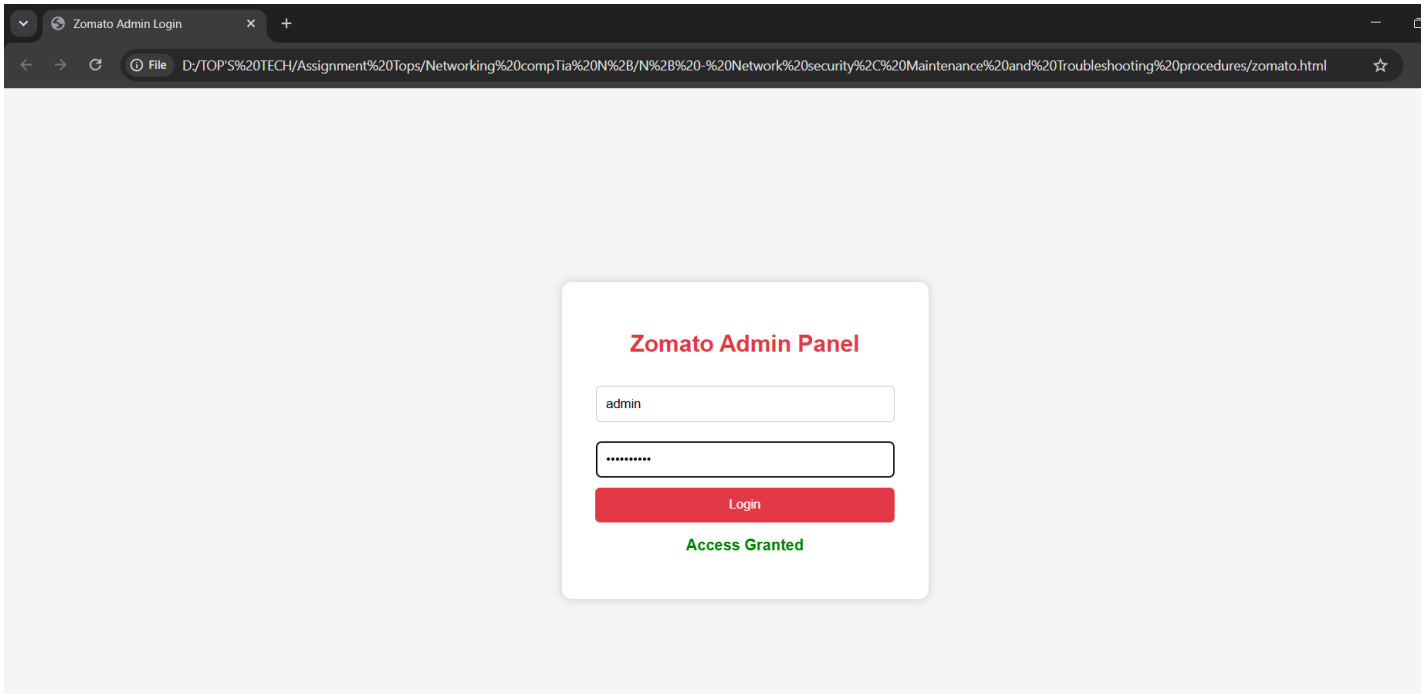
    </script>

</body>
</html>
```

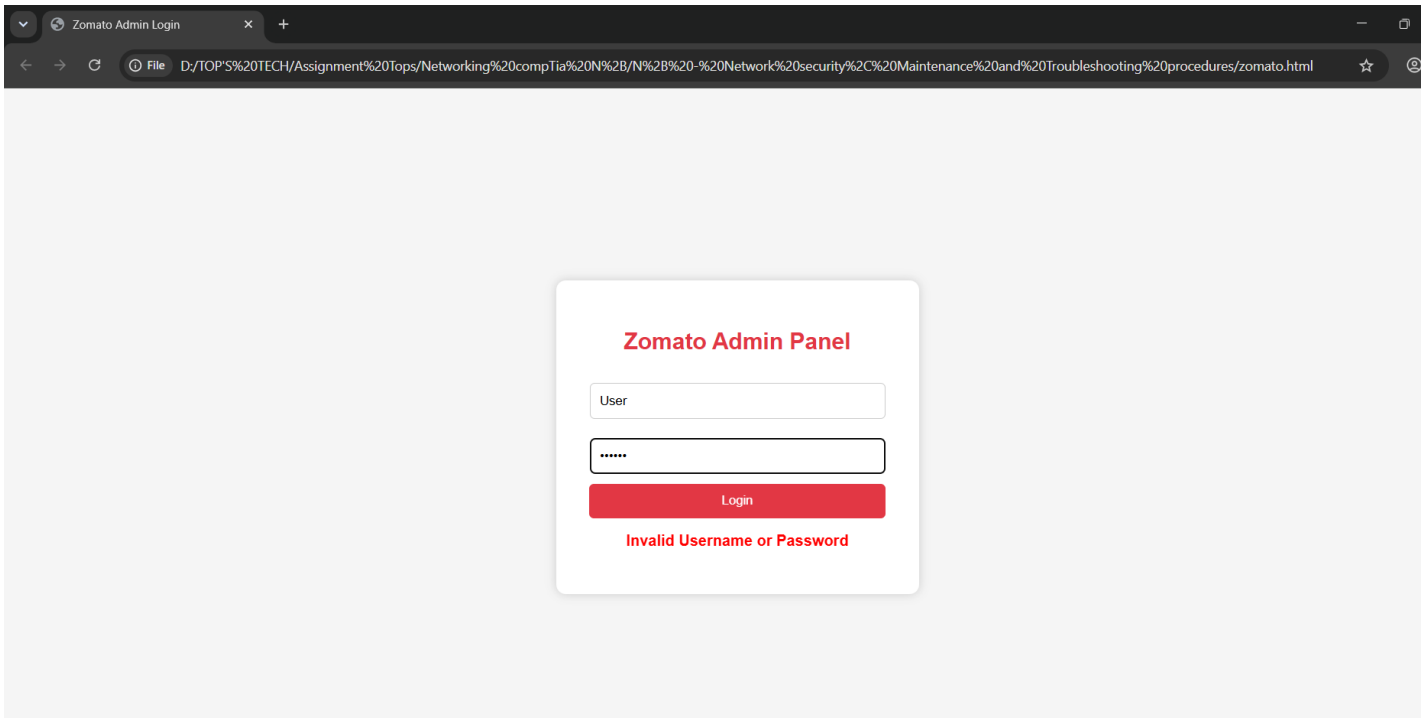
- **Working of the Login Form**

1) The user opens the Zomato Admin Login page in a web browser.
2) The login form asks the user to enter a username and password.
3) When the Login button is clicked, the JavaScript function checkLogin() is executed.
4) The entered username and password are compared with the predefined administrator credentials.
5) If both values match, the message **"Access Granted"** is displayed in green.
6) If either value is incorrect, the message **"Invalid Username or Password"** is displayed in red.

- **Expected Output**
- **Case 1: Correct Login Details**
- Username: admin
- Password: Zomato@123
- **Output:** Access Granted



- **Case 2: Incorrect Login Details**
- Username: user
- Password: 12345
- **Output:** Invalid Username or Password



- **Result**
- A simple Zomato Admin Login Form was successfully created using HTML, CSS, and JavaScript. The form checks the username and password against predefined sample credentials and displays "Access Granted" only when both fields match.

❖ **2. Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Use a dictionary to map roles to allowed resources.**

➢ **Aim**

- To create a Python function named check_access(user_role, resource) that checks whether a user is allowed to access a particular resource based on their assigned role.

➢ **Introduction**

- **Role-Based Access Control (RBAC)** is a security method in which access to system resources is provided according to the role assigned to a user.
- Different users have different permissions depending on their responsibilities. For example, a customer can view order history, while a restaurant owner can edit the menu and manage orders.
- In this assignment, Python dictionaries are used to store the permissions of different user roles. A function checks whether a particular role is allowed to access a requested resource.

➢ **Python Code**

➢ # Dictionary containing roles and their allowed resources

```
permissions = {
    "customer": [
```

```
        "menu_view",
        "order_history"
    ],

    "restaurant_owner": [
        "menu_view",
        "menu_edit",
        "order_history"
    ],

    "admin": [
        "menu_view",
        "menu_edit",
        "order_history",
        "admin_dashboard"
    ]
}


# Function to check access permission

def check_access(user_role, resource):

    if user_role not in permissions:
        return False

    if resource in permissions[user_role]:
        return True

    return False
```

```
        # Testing the function

print("Customer - Menu Edit:",
            check_access("customer", "menu_edit"))

        print("Customer - Order History:",
            check_access("customer", "order_history"))

        print("Restaurant Owner - Menu Edit:",
            check_access("restaurant_owner", "menu_edit"))

        print("Restaurant Owner - Admin Dashboard:",
            check_access("restaurant_owner", "admin_dashboard"))

        print("Admin - Admin Dashboard:",
            check_access("admin", "admin_dashboard"))
```

- **Explanation of the Code**
  1. **Permissions Dictionary:** A Python dictionary named permissions stores user roles and their allowed resources.
  2. **Customer Role:** A customer is allowed to view the menu and access their order history.
  3. **Restaurant Owner Role:** A restaurant owner is allowed to view and edit the menu and access order history.
  4. **Admin Role:** An administrator has permission to access all the resources listed in the dictionary, including the admin dashboard.

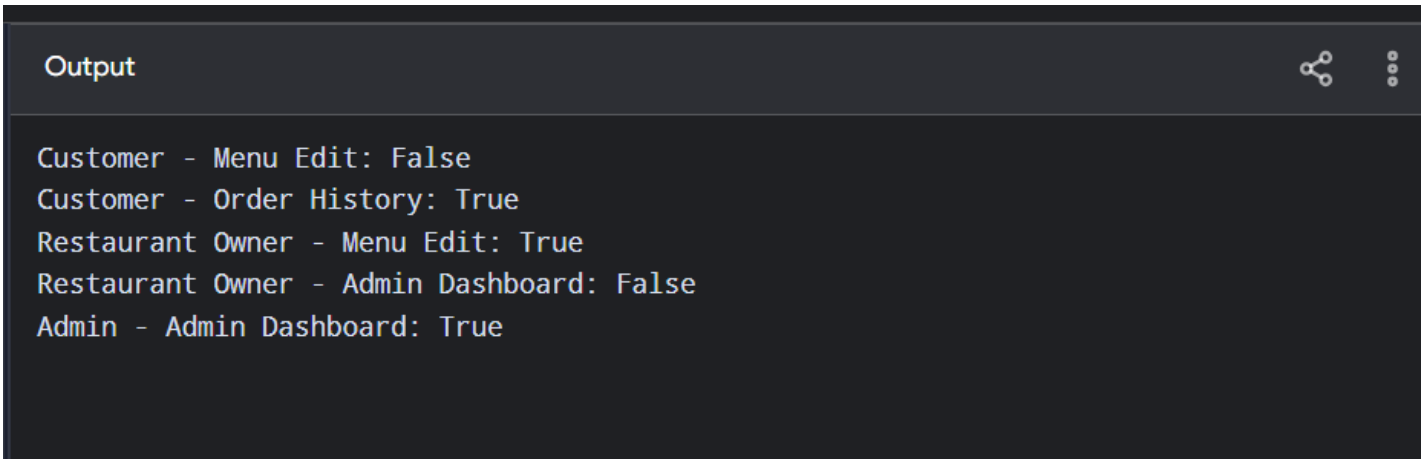
5. **check_access() Function:** This function accepts two parameters: user_role and resource.
6. **Role Verification:** First, it checks whether the given role exists in the dictionary. If the role does not exist, it returns False.
7. **Resource Verification:** It checks whether the requested resource is present in the list of permissions assigned to that role.
8. **Return Value:** If the resource is allowed, the function returns True; otherwise, it returns False.

- **Test Cases and Expected Output**

No.	User Role	Resource	Expected Result
1	customer	menu_edit	False
2	customer	order_history	True
3	restaurant_owner	menu_edit	True
4	restaurant_owner	admin_dashboard	False
5	admin	admin_dashboard	True

- **Expected Output**

Customer - Menu Edit: False
Customer - Order History: True
Restaurant Owner - Menu Edit: True
Restaurant Owner - Admin Dashboard: False
Admin - Admin Dashboard: True



- **Advantages of Role-Based Access Control**
  1. **Improved Security:** Users can access only the resources permitted for their roles.
  2. **Easy Management:** Permissions can be managed through a single dictionary.
  3. **Prevents Unauthorized Access:** Users cannot access restricted resources through this permission check.
  4. **Flexible:** New roles and resources can be added easily.

- **Result**
- The Python function check_access(user_role, resource) was successfully created and tested. It returns True when a user role is permitted to access a resource and False when access is not allowed.

❖ **3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.**

➢ **Aim**

- To create a Python program that simulates a Two-Factor Authentication (2FA) login process like Paytm. The program verifies the user's password, generates a 6-digit OTP, asks the user to enter the OTP, and displays "Login Successful" only when both authentication steps are completed correctly.

➢ **Introduction**

- **Two-Factor Authentication (2FA)** is a security method that requires users to verify their identity using two different authentication steps before accessing an account.
- The first factor is something the user knows, such as a password. The second factor is an additional verification method, such as a One-Time Password (OTP).
- For example, when a user logs in to a payment application like Paytm, the application may ask for a password or other login credentials and then verify the user using an OTP.
- In this assignment, Python is used to simulate this process by generating a random 6-digit OTP and checking whether the entered OTP matches the generated OTP.

➢ 3. Requirements

- Computer or laptop
- Python installed
- Python IDLE or Visual Studio Code
- Basic knowledge of Python

➢ Python Program

```
import random

# Predefined login credentials
correct_password = "Paytm@123"

# Step 1: Ask for password
password = input("Enter your password: ")

if password == correct_password:

    print("Password Verified Successfully!")

    # Step 2: Generate a 6-digit OTP
    otp = random.randint(100000, 999999)

    print("Your OTP is:", otp)

    # Step 3: Ask user to enter OTP
    user_otp = input("Enter the 6-digit OTP: ")

    # Step 4: Verify OTP
    if user_otp == str(otp):
```

```
        print("Login Successful")
    else:
        print("Invalid OTP. Login Failed.")

else:
    print("Incorrect Password. Access Denied.")
```

- **Explanation of the Program**
  1. **Import Random Module:** The random module is imported to generate a random 6-digit OTP.
  2. **Set Password:** A predefined sample password is stored in the variable correct_password.
  3. **Password Verification:** The program asks the user to enter a password and compares it with the stored password.
  4. **OTP Generation:** If the password is correct, random.randint(100000, 999999) generates a random 6-digit OTP.
  5. **OTP Input:** The generated OTP is displayed in the console for this educational simulation, and the user is asked to enter it.
  6. **OTP Verification:** The entered OTP is compared with the generated OTP.
  7. **Login Result:** If both the password and OTP are correct, the program displays "Login Successful". Otherwise, it displays an error message.

- **Output**
- **Case 1: Correct Password and Correct OTP**

```
Output

Enter your password: Paytm@123
Password Verified Successfully!
Your OTP is: 920927
Enter the 6-digit OTP: 920927
Login Successful
```

- **Case 2: Correct Password but Incorrect OTP**

```
Output

Enter your password: Paytm@123
Password Verified Successfully!
Your OTP is: 418156
Enter the 6-digit OTP: 410000
Invalid OTP. Login Failed.
```

- **Case 3: Incorrect Password**

```
Output

Enter your password: payaj
Incorrect Password. Access Denied.
```

➢ **7. Advantages of Two-Factor Authentication**

1. **Improved Security:** It adds an extra layer of protection to user accounts.
2. **Prevents Unauthorized Access:** Knowing only the password is not sufficient to complete the login.
3. **Identity Verification:** The OTP provides an additional verification step.
4. **Account Protection:** It helps reduce the risk of unauthorized account access.

➢ **Result**

➢ The Two-Factor Authentication program was successfully created using Python. The program verifies the password, generates a random 6-digit OTP, and displays "Login Successful" only when the entered password and OTP are correct.

❖ **4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.**

➢ **Aim**

- To write a JavaScript function that filters an array of user objects and returns only those users who have a premium account and have logged in within the last seven days.

➢ **Introduction**

- An array of objects is a data structure used to store multiple records, where each object contains different properties.
- In web applications, user information may include a username, premium membership status, and last login date.
- The JavaScript filter() method is used to create a new array containing only those elements that satisfy a given condition.
- In this assignment, the filter() method is used to select users who:

Have premium membership (isPremium is true).
Have logged in within the last seven days.

➢ **Today's Date:** 2024-06-10

➢ **JavaScript Program**

```
// Array of user objects

const users = [
    {
      username: "Prem",
      isPremium: true,
      lastLogin: "2024-06-09"
    },
    {
      username: "Rahul",
      isPremium: false,
      lastLogin: "2024-06-08"
    },
    {
```

```
        username: "Amit",
        isPremium: true,
        lastLogin: "2024-06-05"
    },
    {
        username: "Neha",
        isPremium: true,
        lastLogin: "2024-05-30"
    },
    {
        username: "Karan",
        isPremium: true,
        lastLogin: "2024-06-10"
    },
    {
        username: "Priya",
        isPremium: false,
        lastLogin: "2024-06-04"
    }
];

// Today's date
const today = new Date("2024-06-10");

// Calculate date seven days before today
const sevenDaysAgo = new Date("2024-06-03");
```

```
// Function to filter premium users
function getPremiumUsers(users) {

    return users.filter(user => {

        const loginDate = new Date(user.lastLogin);

        return user.isPremium === true &&
               loginDate >= sevenDaysAgo &&
               loginDate <= today;
    });
}

// Call the function
const result = getPremiumUsers(users);

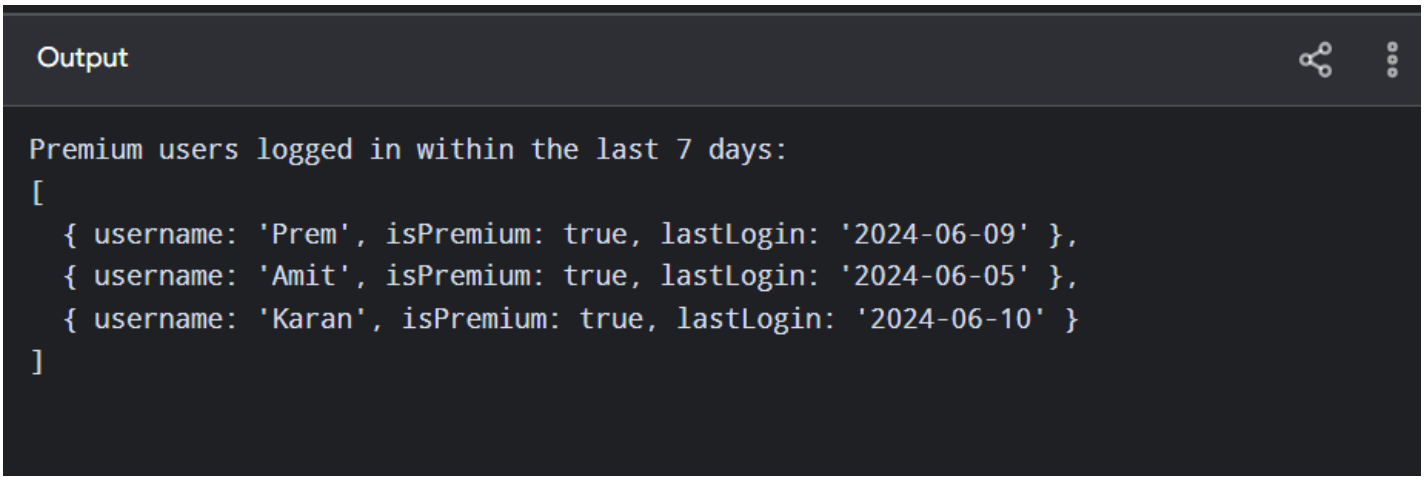
// Display the result
console.log("Premium users logged in within the last 7 days:");

console.log(result);
```

- **Explanation of the Program**
  1) **Array of Users:** An array named users is created containing user objects with three properties: username, isPremium, and lastLogin.
  2) **Today's Date:** The current date for this example is set to 2024-06-10.
  3) **Date Calculation:** The date 2024-06-03 is selected as the starting date for the seven-day period.

4) **Function Creation:** The function getPremiumUsers(users) is created to filter the user records.
5) **Filter Method:** The JavaScript filter() method checks each user object in the array.
6) **Premium Condition:** user.isPremium === true checks whether the user has premium membership.
7) **Date Condition:** The login date must be on or after 2024-06-03 and on or before 2024-06-10.
8) **Return Result:** Only users who satisfy both conditions are returned in a new array.

- **Expected Output**
- Premium users logged in within the last 7 days:



- **Result**
- The JavaScript function was successfully created and tested. It returns only those users who have premium membership and whose last login date falls within the specified seven-day period.

❖ 5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

➢ Aim

- To identify five common access control mistakes made by developers in real-world applications such as Instagram and Flipkart and explain how these mistakes can be prevented.

➢ Introduction

- Access Control is a security mechanism that determines which users are allowed to access specific data, features, and resources in an application.
- Applications such as Instagram and Flipkart contain sensitive information, including user profiles, personal details, orders, payment information, and administrative features.
- If access control is not implemented properly, unauthorized users may access information or perform actions that they are not permitted to perform.

➢ Five Common Access Control Mistakes and Their Prevention Tips

➢ 1. Missing Authorization Checks

Mistake: Developers sometimes verify whether a user is logged in but fail to check whether that user has permission to access a particular resource.

Example: A normal Flipkart customer might gain access to an administrative feature if proper authorization checks are missing.

Prevention: Always verify the user's role and permissions on the server before allowing access to protected resources.

- **2. Insecure Direct Object References (IDOR)**
  Mistake: Applications may rely only on an object ID, such as an order ID or user ID, without checking whether the requested object belongs to the logged-in user.
  Example: A customer might access another customer's order information if ownership is not verified.
  Prevention: Check resource ownership and user permissions on every request before returning private information.

- **3. Excessive User Permissions**
  Mistake: Developers sometimes give users more permissions than they actually need.
  Example: A normal Instagram user might receive access to administrative functions because permissions are configured incorrectly.
  Prevention: Follow the Principle of Least Privilege by giving users only the minimum permissions required for their tasks.

- **4. Improper Session Management**
  Mistake: Applications may fail to properly manage login sessions, allowing unauthorized access through expired or improperly protected sessions.
  Example: A user may remain logged in on a shared device if the application does not handle logout and session expiration correctly.

Prevention: Use secure session cookies, expire inactive sessions, and invalidate sessions properly when users log out.

- **5. Client-Side Only Access Control**
  Mistake: Developers sometimes hide buttons or pages in the frontend but do not enforce the same permissions on the backend.
  Example: An administrative button may be hidden from ordinary users, but the related server operation may still be accessible without proper authorization.
  Prevention: Enforce all important access control checks on the server instead of relying only on frontend restrictions.

- **Summary Table**

No.	Common Mistake	Prevention Tip
1	Missing Authorization Checks	Verify user permissions on the server.
2	Insecure Direct Object References (IDOR)	Check resource ownership for every request.
3	Excessive User Permissions	Follow the Principle of Least Privilege.
4	Improper Session Management	Use secure sessions and proper expiration.

5	Client-Side Only Access Control	Enforce authorization on the backend.

- **Result**
- Five common access control mistakes in real-world applications were identified, and suitable prevention tips were provided for each mistake.